



Document Summary

This note introduces the fundamentals of Natural Language Processing (NLP), covering its definition, evolution from traditional to deep techniques, and key applications like sentiment analysis, machine translation, and text summarization. It explores text structures, preprocessing steps including tokenization and stemming, and advanced techniques such as syntactic parsing and named entity recognition. The content emphasizes interdisciplinary connections and practical implementations, providing a foundational overview for NLP studies.

Introduction to Natural Language Processing Fundamentals

Lecture Overview

- **Source:** Politecnico di Torino, Prof. Luca Cagliero, Dipartimento di Automatica e Informatica.
- **Goals:**
 - Introduction to [Natural Language Processing](#).
 - Overview of main objectives, techniques, and application scenarios.
 - Description of traditional NLP fundamentals.
 - NLP pipeline and text preprocessing.

To begin, this lecture provides a foundational understanding of NLP by first defining the field and then exploring its evolution, related concepts, and practical applications. This structured approach ensures a logical progression from basics to advanced topics.

Definition of Natural Language Processing



Natural Language Processing (NLP)

Aims at enabling computers to process human languages intelligently. In essence, NLP bridges the gap between human communication and machine understanding.

- Tight connection between fields in computer science and linguistics: [Artificial Intelligence](#), [Machine Learning](#), Cognitive science, Information processing, and more. These interdisciplinary links allow NLP to draw on diverse methodologies for robust language handling.
- NLP studies the connection between machine perception and languages. For example, it examines how machines can "perceive" nuances like sarcasm or context in text.
- Programs computers to fruitfully process large natural language corpora. This involves analyzing vast collections of text data to extract meaningful insights.

- Applications: Language understanding, Machine translation, Text generation, and more. These applications demonstrate NLP's versatility in real-world scenarios.
- **Reference:** Manning C.D., Schuetze H. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999. ISBN 9780262133609.
- Additional resource: <https://chat.openai.com/>.

Traditional vs. Deep Natural Language Processing



Traditional NLP techniques

Relied on ad hoc linguistic models, e.g., Syntactic rules, Dictionaries, Text generation, and more; does not require automated learning. These rule-based methods provide explicit control but can be rigid and labor-intensive to develop.



Deep NLP techniques

Leverage [Machine Learning](#) models to automate learning. In contrast, deep approaches learn patterns directly from data, adapting more flexibly to complex language variations.

This distinction highlights how NLP has evolved from manual rule crafting to data-driven automation, improving scalability and accuracy.

#nlp/traditional-vs-deep

Related Buzzwords

Text Mining



Text Mining

Derives significant information from text, focusing on unstructured and semi-structured text. It transforms raw text into actionable knowledge by uncovering hidden patterns.

- Encompasses: [Information Retrieval](#), [Machine Learning](#), [Natural Language Processing](#), Knowledge management (KM). These components work together to handle the full spectrum of text analysis tasks.
- **Reference:** *The Text Mining Handbook: Advanced Approaches in Analyzing Unstructured Data*. Ronen Feldman, James Sanger. 2007. ISBN: 9780521836579.

Natural Language Understanding (NLU/NLI)



Natural Language Understanding or Interpretation (NLU/NLI)

NLP subtopic for machine reading comprehension using AI techniques. NLU focuses on interpreting the intent and meaning behind text, going beyond surface-level processing.

- Encompasses: Text categorization, Entity Recognition and Disambiguation, and more. These tasks enable machines to grasp context and relationships in language.
- **Reference:** *The Text Mining Handbook: Advanced Approaches in Analyzing Unstructured Data.* Ronen Feldman, James Sanger. 2007. ISBN: 9780521836579.

Natural Language Generation (NLG)



Natural Language Generation (NLG)

Generates human-like text or speech from structured data or instructions. NLG reverses the understanding process by creating coherent outputs from inputs like databases or commands.

- Encompasses: Summarization, Machine translation, Question Answering, and more. These applications showcase NLG's role in producing natural-sounding responses.
- **Reference:** *The Text Mining Handbook: Advanced Approaches in Analyzing Unstructured Data.* Ronen Feldman, James Sanger. 2007. ISBN: 9780521836579.

#nlp/related-concepts

Overview of NLP Applications

- NLP techniques apply in knowledge discovery, decision support systems, and data science pipelines. By integrating NLP, these systems can handle textual data effectively, enhancing decision-making and insights.
- **Reference:** *The Text Mining Handbook: Advanced Approaches in Analyzing Unstructured Data.* Ronen Feldman, James Sanger. 2007. ISBN: 9780521836579.
- Focus areas: [Sentiment Analysis](#), Text categorization, [Machine Translation](#), Question answering, Topic modelling, Text summarization.

The following sections delve into these key applications, illustrating their mechanisms and real-world relevance.

#nlp/applications

Sentiment Analysis



Sentiment Analysis

Extracts writer's feelings, opinions, emotions, likes/dislikes; also known as opinion mining. This technique quantifies subjective information in text to gauge overall polarity (positive, negative, neutral).

- Identifies opinions/human behavior from plain text, using traditional NLP rules or [Machine Learning](#). Traditional rules might use keyword matching, while ML models learn from annotated data for better nuance detection.
- Use Cases:** Hotel review analysis, News trading, Hate speech detection, Advertisement placing, and more. For instance, in hotel reviews, it can aggregate customer satisfaction scores.



Example of Sentiment Analyzer: VADER

- Input:** `sentiment_analyzer_scores("The phone is super cool.")`
- Output:** `The phone is super cool --- {'neg': 0.0, 'neu': 0.326, 'pos': 0.674, 'compound': 0.7351}`
- Resource:** <https://github.com/cjhuto/vaderSentiment>.

To clarify VADER's usage, consider this simple Python implementation:

```
1  from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
2
3  # Initialize the sentiment analyzer
4  analyzer = SentimentIntensityAnalyzer()
5
6  # Example text
7  text = "The phone is super cool."
8
9  # Compute sentiment scores
10 scores = analyzer.polarity_scores(text)
11 print(scores) # Output: {'neg': 0.0, 'neu': 0.326, 'pos': 0.674, 'compound': 0.7351}
```

This code demonstrates how VADER processes text to output compound scores ranging from -1 (most negative) to 1 (most positive).

Text Categorization



Text Categorization

Assigns predefined labels to documents or text snippets; also known as text classification. It automates the organization of text based on content themes.

- Supervised process: Learns from labeled documents/text snippets to label new unlabeled ones. During training, models identify patterns in labeled data to predict categories for unseen text.
- If multiple labels allowed: Multi-label text classification. This handles cases where text fits multiple categories, like a news article on politics and economy.
- Relies on traditional NLP rules or [Machine Learning](#). Rules might use keyword thresholds, while ML employs algorithms like Naive Bayes.
- **Use Cases:** Spam detection, Ticket management, Automatic text annotation, Document management, and more. Spam detection, for example, classifies emails as "spam" or "not spam" to filter inboxes.

#nlp/text-categorization

Machine Translation



Machine Translation

Automatically translates text or speech between languages. It enables cross-lingual communication by converting source text into a target language while preserving meaning.

- **Approaches:** Rule- or dictionary-based, Statistical methods, [Deep Learning](#). Each approach builds on linguistic knowledge but differs in automation and accuracy.
- **Resource:** memoq (latest access: April 2021). <https://translate.google.com> (latest access: April 2021).

Rule-Based Machine Translation

- Uses linguistic information from source/target languages, retrieved from bilingual dictionaries and grammars. This method applies predefined rules to map structures directly.
- Also known as Knowledge-Based Machine Translation or Classical MT Approach. It excels in controlled domains but struggles with idiomatic expressions.

Statistical Machine Translation

- Based on statistical models with parameters from bilingual text corpora analysis. Models learn probabilities from parallel texts to predict translations.
- **Key Ideas:** Every sentence in one language possibly translates any in the other; selects highest-probability translation. This probabilistic selection favors common alignments.
- **Reference:** *Introduction to Statistical Machine Learning*. Raghav Bhashyal.

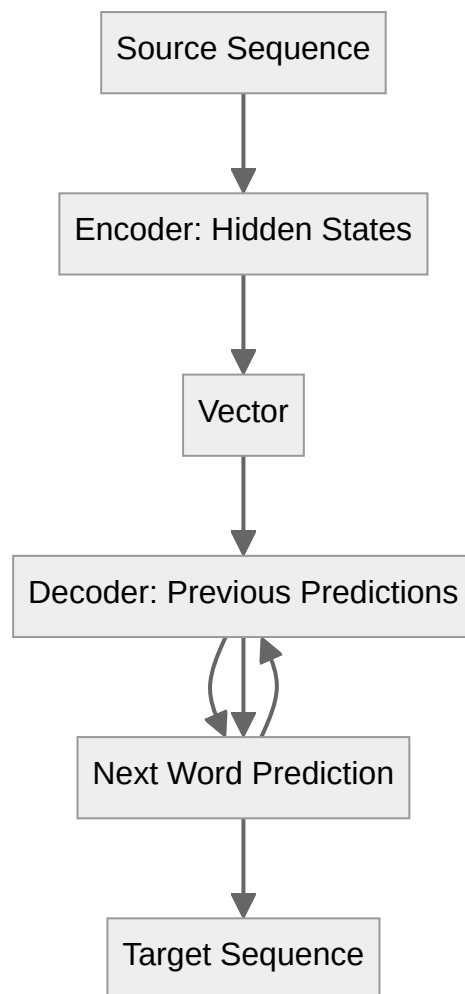
Neural Machine Translation

- Encoder creates fixed-size context vector from all hidden states. The encoder processes the input sequence into a compact representation capturing the entire context.
- Decoder uses context vector and previous predictions to generate target sequence. It autoregressively builds the output, attending to relevant input parts.
- Decoder predicts next word given context vector and prior words. Attention mechanisms enhance focus on important source elements.



Visual Overview of Neural Machine Translation Process

For a visual overview of the Neural Machine Translation process, the following diagram illustrates the encoder-decoder architecture:



#nlp/machine-translation

#level/intermediate

Question Answering



Question Answering

Information Retrieval area providing relevant answers to natural language questions. It simulates human-like responses by searching and extracting pertinent information.

- **Main Steps:** Question classification, Information Retrieval, Answer extraction. Classification identifies question type (e.g., who, what), retrieval fetches candidates, and extraction refines the final answer.
- **Key Properties:**
 - **Source:** Web pages, knowledge bases, social data collections, etc. Diverse sources expand the system's knowledge base.
 - **Question Formulation:** Domain-specific, factoid, etc. Factoid questions seek specific facts, while domain-specific ones require expert knowledge.

- **Answer Type:** Word, sentence, paragraph, image, video, etc. Flexibility in output format improves usability.
- **Practical Use Cases:** Chatbots, Remote medical assistance, and more. Chatbots, for example, provide instant customer support.
- **Open-Domain Question Answering:** Specific variant. It handles broad queries without domain restrictions.

Conversational AI

- QA extension/adaptation. It builds on QA by incorporating dialogue dynamics.
- Considers interaction history and context, not treating questions separately. This maintains coherence across turns, like remembering prior user inputs.
- Preserves event hierarchy and order. Chronological tracking ensures logical conversation flow.

#nlp/question-answering

#ai/conversational

Topic Modelling



Topic Modelling

Unsupervised ML techniques detecting key word/phrase patterns in document collections. It reveals latent themes without labeled data, grouping similar content.

- Topic model: Statistical representation of abstract word groups characterizing documents. Topics are distributions over words, and documents over topics.
- **Main Techniques:** Latent Semantic Indexing (LSI), Latent Dirichlet Allocation (LDA), Aspect modelling. LDA, for instance, assumes documents are mixtures of topics.
- **Practical Use Cases:** Information retrieval, Data visualization, and more. In retrieval, it improves search by matching query topics to documents.
- **Dimensionality Reduction:** Data on/near low d-dimensional subspace; subspace axes represent data effectively. This compresses high-dimensional text into interpretable lower dimensions.
- **Reference:** *Mining of Massive Datasets*. Jure Leskovec, Anand Rajaraman, Jeffrey D. Ullman. 2nd Ed. Cambridge University Press, 2014. ISBN 978-1107077232.
- **Resource:** Topic Modeling - BERTopic. <https://github.com/MaartenGr/BERTopic>.



Illustration of LDA's Topic Extraction

To illustrate LDA's topic extraction, consider this simplified Python example using scikit-learn:


```

1  from sklearn.decomposition import LatentDirichletAllocation
2  from sklearn.feature_extraction.text import CountVectorizer
3
4  # Sample documents
5  documents = ["NLP is fascinating.", "Machine learning advances NLP.", "Tex
6
7  # Vectorize text
8  vectorizer = CountVectorizer()
9  X = vectorizer.fit_transform(documents)
10
11 # Fit LDA model with 2 topics
12 lda = LatentDirichletAllocation(n_components=2, random_state=0)
13 lda.fit(X)
14
15 # Display top words per topic
16 feature_names = vectorizer.get_feature_names_out()
17 for topic_idx, topic in enumerate(lda.components_):
18     top_words = [feature_names[i] for i in topic.argsort()[-3:]]
19     print(f"Topic {topic_idx}: {top_words}")

```

This code generates topics like ["nlp", "is", "fascinating"] and ["machine", "learning", "advances"], highlighting word associations.

#nlp/topic-modelling

#level/intermediate

Text Summarization



Text Summarization

Shortens large textual document collections into concise summaries of salient content. It condenses information while retaining core ideas.

- **Reference:** *Multimodal Abstractive Summarization for How2 Videos*. Shruti Palaskar, Jindrich Libovicky, Spandana Gella, Florian Metze. <https://arxiv.org/abs/1906.07901>.
- **Input:** Single document, Multi-document collection, Multimodal data. Multi-document inputs merge insights from multiple sources.
- **Output:**
 - **Extractive Summary:** Picks existing content. It selects key sentences directly from the original text.

- **Abstractive Summary:** Generates new content. It paraphrases and synthesizes information for more natural summaries.
- **Approach:**
 - **Supervised:** e.g., neural summarization. Uses labeled data to train models for summary generation.
 - **Unsupervised:** e.g., itemset mining, clustering, Latent Semantic Analysis, graph ranking. These rely on patterns like frequency or similarity.
- **Language:** Single-language, Multi-lingual, Cross-lingual. Cross-lingual handles summaries across languages.
- **Domain Specificity:** General-purpose, Context-dependent, Query-driven. Query-driven tailors summaries to user questions.
- **Time Dependency:** Temporal summarization, Timeline summarization, Incremental summarization. Timeline versions evolve with events.
- **Practical Use Cases:** Content curation, Accessibility, Learning analytics, Highlight extraction. For accessibility, summaries aid quick comprehension for visually impaired users.
- **Examples of Summarization Methods:** TimeLine Summarization of news articles.
- **Reference:** *Summarize Dates First: a paradigm shift in timeline summarization*. Moreno La Quatra, Luca Cagliero, Elena Baralis, Maurizio Montagnulo, Alberto Messina. ACM SIGIR 2021.

#nlp/text-summarization

Text Structure

- Text divides into units for automated processing and knowledge extraction. Understanding these units is crucial for breaking down complex texts into manageable components.
- Aggregation levels vary by objective and language. For example, analysis goals might emphasize words for semantics or sentences for syntax.

#nlp/text-structure

Basic Units



Character

Smallest text unit (letter, digit, space, special character); low utility without positional info (e.g., spaces separate words, full stops end sentences, question marks indicate interrogatives). Positional context reveals structure, like punctuation signaling sentence boundaries.



Word

Series of letters between spaces; smallest meaningful language unit for target audience, assuming reader agreement. Words carry core semantic load in language.



N-gram

Contiguous sequence of N textual units (phonemes, syllables, letters, words); syntactic/semantic relevance not guaranteed; unigram (1-gram), bigram (2-gram), etc. N-grams capture local patterns, useful for probability models despite lacking full context.



Multi-word Expressions

Textual forms of at least two lexical items. These fixed phrases, like "kick the bucket," convey idiomatic meanings.



Sentence

Text snippet separated by punctuation (e.g., full stop, question mark, exclamation mark). Sentences form complete thoughts, aiding parsing.



Phrase

Sentence part of grouped words without subject/verb (e.g., "after dinner", "waiting for the bus"); incomplete thought. Phrases build sentence structure modularly.



Clause

Sentence part with subject and predicate; independent (e.g., "the dog barked at him") or subordinate (e.g., "when the old man came") for added insights. Clauses add depth, with subordinates providing conditions or details.



Lexical Chain

Sequence of related words spanning short (adjacent) or long (entire text) distances. These chains maintain topic coherence across text.



Lemma

Canonical dictionary form of word/multi-word expression from candidate forms. It standardizes variations like "running" to "run."

Lexeme

Set of words/multi-word forms with same meaning; lemma represents it conventionally (e.g., inflected endings). Lexemes group synonyms and inflections.

Stem

Base word form (like lemma) not from dictionary; uses static rules for inflections. Stemming is faster but less precise than lemmatization.

Bag-Of-Word (BOW)

Unordered word set without repetitions. BOW ignores order, treating text as a multiset of words for simple representation.

- **Examples of N-grams:** Source: <https://deeptai.org/> (latest access: April 2021).

Generating Bigrams as an Example of N-grams

For generating bigrams as an example of N-grams, here is a Python snippet using NLTK:

```
1  from nltk import bigrams
2  from nltk.tokenize import word_tokenize
3  import nltk
4  nltk.download('punkt') # Download tokenizer if needed
5
6  # Example text
7  text = "Natural language processing is fascinating"
8  tokens = word_tokenize(text.lower())
9
10 # Generate bigrams
11 bigrams_list = list(bigrams(tokens))
12 print(bigrams_list) # Output: [('natural', 'language'), ('language', 'pro
```

This code tokenizes text and extracts adjacent word pairs, illustrating local sequence capture.

- **Examples of Lexical Chains:** Source: <https://Wikipedia.org> (latest access: April 2021).

- **Examples of Clauses and Phrases:** Source: <https://knowitinfo.com/> (latest access: April 2021).

Higher-Level Structures

- **Paragraph:** Sequence of sentences; partitionable into subparagraphs. Paragraphs group related ideas for thematic cohesion.
- **Section:** Consecutive paragraphs; partitionable into subsections/subsubsections. Sections organize content hierarchically.
- **Text Structure Sectioning.** This process aids navigation and analysis by imposing logical divisions.

Text Preprocessing: The Taxonomy

- **Reference:** *The Text Mining Handbook: Advanced Approaches in Analyzing Unstructured Data.* Ronen Feldman, James Sanger, 2007. ISBN: 9780521836579.

Preprocessing prepares raw text for analysis by standardizing and cleaning it, ensuring consistency across models.

#nlp/preprocessing

#level/beginner

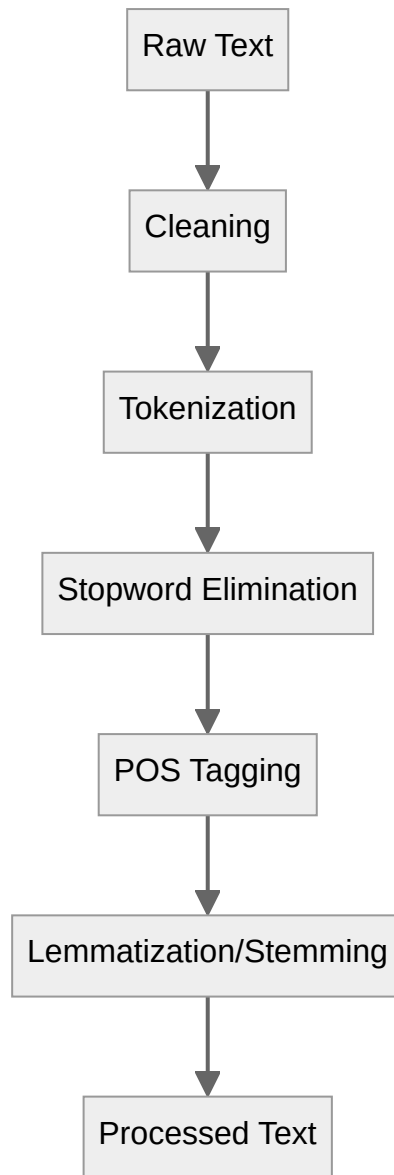
Text Preprocessing Steps

1. **Cleaning:** Filters noise, errors, redundant content. It removes irrelevant elements to focus on meaningful data.
2. **Tokenization:** Divides raw text into units/sub-units. This segmentation enables granular processing.
3. **Stopword Elimination:** Removes frequent words with little semantic meaning. It reduces noise from common fillers.
4. **Part-Of-Speech Tagging:** Annotates words with sentence roles. POS provides syntactic context.
5. **Lemmatization and Stemming:** Maps inflections/derivatives to canonical forms. These normalize variations for uniformity.



NLP Preprocessing Pipeline Visualization

The NLP preprocessing pipeline can be visualized as a sequential workflow:



Text Cleaning

- Removes special characters/extra spaces (unrecognized symbols, OCR errors, etc.). This handles artifacts from data collection.
- Case normalization (upper/lower cases). Uniform casing prevents duplicates like "Apple" vs. "apple."
- Data format conversion (e.g., HTML, XML, JSON, CSV; may need schema). Parsing formats extracts pure text.
- Handles non-textual content. Images or tables are often stripped or converted.

Language Dependence in Preprocessing

- Affects: Cleaning (partially), Tokenization, Stopword elimination, Part-Of-Speech tagging, Lemmatization/stemming. Language-specific rules ensure accuracy.

- **Challenge:** Unknown source language. Detection tools can mitigate this, but errors propagate.

Tokenization

- Breaks continuous character stream into meaningful constituents. It identifies boundaries for words or sentences.
- Divides text into units/sub-units; identifies sentence boundaries via punctuation/capitalization. Rules vary, e.g., handling contractions.
- Strongly language-dependent. English uses spaces, while Chinese relies on characters.
- **Source:** <https://kddnuggets.com> (latest access: April 2021).
- **Example of Tokenizer:** Source: https://keras.rstudio.com/reference/text_tokenizer.html (latest access: April 2021).



Basic Tokenization Example in Python

A basic tokenization example in Python:

```
1  from nltk.tokenize import word_tokenize, sent_tokenize
2  import nltk
3  nltk.download('punkt')
4
5  # Example text
6  text = "NLP is great. It processes text intelligently."
7
8  # Word and sentence tokenization
9  words = word_tokenize(text.lower())
10 sentences = sent_tokenize(text)
11 print(words) # Output: ['nlp', 'is', 'great', '.', 'it', 'processes', 'te
12 print(sentences) # Output: ['NLP is great.', 'It processes text intellige
```

This snippet splits text into words and sentences, foundational for further analysis.

Stopword Elimination



Stop Words

Most common language words conveying limited info (e.g., prepositions, articles, conjunctions). Examples include "the," "is," "and."

- Filtered before processing. Removal streamlines models by focusing on content words.

- **Source:** <https://kddnuggets.com> (latest access: April 2021).
- Language-dependent; some Deep NLP models skip it. Transformers like [BERT](#) handle stopwords contextually.
- No universal stopwords list; customizable (e.g., search engines vary; some remove lexical words like "want" for performance). Customization tailors to domain needs.
- **Example of Stopword List:** Source: <https://gist.github.com/sebleier/554280> (latest access: April 2021).
- **Example of Text Preprocessing Pipeline:** Source: www.meaningcloud.com (latest access: April 2021).
- **Caveat:** Removing "not" alters meaning. Negations carry semantic weight, so careful handling is essential.



Caveat on Stopword Elimination

For semantics, deprecate stopwords elimination; use Deep NLP. Modern models preserve context better.

Part-Of-Speech Tagging



Part-Of-Speech (POS)

Word category by sentence role. Categories include noun (person/place), verb (action), etc.



POS Tagging

Labels text words with POS (e.g., article, noun, verb, adjective, preposition, number, proper noun). It disambiguates based on neighbors.

- Context/language dependent; on morphological analysis. Morphology examines word forms for clues.
- **Example:** Source: <https://dataingovernment.blog.gov.uk/> (latest access: April 2021).

Lemmatization



Lemmatization

Determines word lemma by intended meaning; groups inflected forms under common lemma. It considers context, e.g., "better" to "good."

- Performs with POS tagging; analyzes sentence/document context (e.g., article, noun, verb, adjective, preposition, number, proper noun). POS refines lemma choice.
- Context/language dependent; relies on morphological analysis. Dictionaries and rules ensure precision.

Stemming



Stemming

Simplified lemmatization finding stem, not lemma. It chops suffixes heuristically, e.g., "running" to "run."

- Lower computational complexity; approximated but acceptable results. Speed suits large-scale processing.
- **Source:** <https://devopedia.org/> (latest access: April 2021).
- **Stemming vs. Lemmatization:** Source: <https://dataingovernment.blog.gov.uk/> (latest access: April 2021).



Comparison of Stemming and Lemmatization

Word	Stem (Porter)	Lemma
Running	Run	Run
Better	Better	Good
Geese	Gees	Goose

This table shows stemming's approximations versus lemmatization's accuracy.

Advanced NLP Techniques

These techniques build on preprocessing to analyze deeper linguistic structures, enhancing comprehension.

#nlp/advanced

#level/advanced

Syntactic Parsing



Syntactic Parsing

Analyzes text syntax per grammar theory. It constructs parse trees representing grammatical hierarchy.

- **Constituency Grammars:** Describe sentence structure via recursively built phrases (syntactically grouped elements). Phrases nest, like noun phrases within sentences.
- **Dependency Grammars:** Analyze word dependencies (e.g., adjective on noun). Edges link heads to modifiers.
- High computational complexity; low quality in real contexts. Ambiguities and long sentences challenge parsers.
- **Parsing Example:** "book that flight". Possible parses: verb "book that flight" or noun "book that flight."
- **Reference:** *Integration Of Supra-Lexical Linguistic Models With Speech Recognition Using Shallow Parsing And Finite State Transducers*. Xiaolong Mou, Stephanie Seneff, Victor Zue. MIT press. 2003.

Shallow Parsing



Shallow Parsing

Simplified parsing. It avoids full trees, focusing on chunks like noun phrases.

- Analyzes unambiguous text snippets (e.g., simple noun/verb phrases). This targets surface patterns efficiently.
- Balances complexity and accuracy. Useful for quick extractions without exhaustive analysis.

Named Entity Recognition (NER)



Named Entity Recognition

Locates/classifies text names. It tags entities in context.

- Identifies entity references (semantically rich ontology-based concepts: persons, organizations, locations, numeric expressions like time/date/money/, proteins, etc.). Ontologies provide semantic grounding.
- **Reference:** *Named Entity Recognition and the Stanford NER Software*. Jenny Rose. Stanford University. 2007.

Word Sense Disambiguation



Word Sense Disambiguation

Identifies word sense in sentence among multiple meanings. It resolves polysemy, e.g., "bank" as river or finance.

- **Approaches:** Knowledge base exploration, [Machine Learning](#), or combination. Knowledge bases like WordNet link senses; ML uses context vectors.
- **Acronym NERD:** "Named Entity Recognition and Disambiguation". NERD extends NER with sense resolution.
- **Reference:** *Named Entity Recognition and the Stanford NER Software*. Jenny Rose. Stanford University. 2007.
- **Word Disambiguation Example:** "cars" (same query yields different Google results). Context determines if automotive or train cars.

Text Representation



Text Representation

Transforms raw text into structured form for [Machine Learning](#)/data mining algorithms (requiring fixed-size relational/key-value models). Variable-length text must map to fixed vectors or tables.

- **Main Issue:** From unstructured/semi-structured to structured form. This conversion enables algorithmic processing.

Types of Text Documents

- **Fully Unstructured Document:** Raw text without sectioning/paragraphs. Pure narrative, like emails.
- **Weakly Structured Document:** Text in sections/paragraphs per predefined format. Reports with headings.
- **Semi-Structured Document:** Text with tags/markups (e.g., XML, HTML). Web pages with metadata.

Feature-Value Data Model

- Structured representation of key text properties (e.g., occurring words, sentences). It uses rows for documents and columns for features.
- **Example:** Column-based DBs. Queries extract patterns efficiently.
- **Main Steps:**
 1. **Feature Engineering:** Defines features/values considering textual units/domain heuristics. Tailors to task, e.g., TF-IDF for relevance.
 2. **Data Transformation:** Builds structured text representation with features. Applies encoding like one-hot for categories.

Feature Engineering

- **Dimensionality Reduction:** Reduces features for high-dimensional data issues. Text often has thousands of unique words, leading to sparsity.
 - **Main Methods:** Latent Semantic Indexing, Latent Dirichlet Analysis, [Principal Component Analysis](#). LSI uncovers semantic links via SVD.
- **Feature Selection:** Reduces features to boost ML performance (e.g., sentiment analysis, classifiers). Eliminates noise for better generalization.

Feature Selection Methods

- **Unsupervised Methods:** Ignore target (remove redundants, e.g., Pearson correlation). Correlation measures feature similarity.
- **Supervised Methods:** Use target (remove irrelevants). Leverages labels for relevance.
 - **Wrapper:** Searches well-performing feature subsets. Tests subsets via model accuracy, computationally intensive.
 - **Filter:** Selects features by target relationship. Scores like chi-squared for quick ranking.
- **Statistical Methods.** Use tests like t-tests for significance.
- **Feature Importance Methods:**
 - **Intrinsic Methods:** Automatic selection during training (e.g., Decision Trees). Trees split on informative features.